NAVCORE-SoOP

Order to first flight

The complete runbook, with an audit of what is still manual, soldered by hand, or not yet measured

---

Board	46.5 × 47.2 mm, 6-layer, STM32H743VIT6, Revision D
Gate verdict	**READY TO ORDER** — every gated prerequisite proven (the gate prints the count)
Hardware verdict	**NOT READY TO FLY** — 6 items need hardware, none of which can change what is orde
Toolchain	KiCad 9.0.8, ArduPilot Copter-4.7.0, Python 3 + pcbnew

*Every figure in this document is derived from the repository's own tooling rather than copied from another document. Where a number is a judgement, an assumption or genuinely unknown, it says so. Sections marked **flag** are the ones capable of costing money, a board, or a bench session.*

Contents

1 How to use this, and how to verify it **3**
1.1 The four commands that matter . . . 3
1.2 Freshness . . . 3

2 Part 1 — Order the bare board and its assembly **3**
2.1 Regenerate, do not trust . . . 3
2.2 What to upload . . . 3
2.3 Fabrication settings . . . 4
2.4 Quantity, and the tier decision . . . 4
2.5 The variant files, and why two of them are identical . . . 4
2.6 Parts worth checking before you pay . . . 5
2.7 The DNP list — derived, never typed . . . 5
2.8 Check these on screen at upload . . . 5
2.9 Paying . . . 5

3 Part 2 — Order the parts: the board's bench first, then Iridium, then the aircraft **5**
3.1 The one order — nothing here should make you order twice . . . 8
3.2 The bench measurements that could have cost you a purchase . . . 9

4 Part 3 — While you wait (free, and the highest-value work) **10**

5 Part 4 — On arrival: inspect before you spend more **11**
5.1 Visual inspection . . . 11
5.2 Confirm the machine-readable facts . . . 11
5.3 The frame measurements that settle a real decision . . . 11
5.4 The two things no drawing can answer . . . 11
5.5 T0 — the mechanical checklist, before any loom is cut . . . 11

6 Part 5 — Mechanical assembly **12**
6.1 Stack height . . . 12
6.2 The two screw warnings — read both . . . 12
6.3 Screw heads against copper . . . 12
6.4 Sequence . . . 13
6.5 What plugs in where . . . 13
6.6 Attaching the Iridium antenna . . . 15
6.7 The downward radar — research first, then this . . . 15
6.8 The harness — six runs, and how not to make a nest of them . . . 16
6.9 Things that will bite you . . . 16
6.10 J2, the ESC connector — confirmed rather than assumed . . . 17

7 Part 6 — Soldering: the buzzer and the LED strip **17**

8 Part 7 — T1: first power, before the MCU sees anything **17**

9 Part 8 — T2: MCU alive **18**

10 Part 9 — T3: sensors on the bench, props OFF **19**
10.1 Antenna feed monitor bench test . . . 20

11 Part 10 — T3a: thermal, and the board measures half of it for you **20**

12 Part 11 — T3b: the RF chain, and the single open question **21**
12.1 Step 1 — record the sky with the bench kit (no board needed) . . . 22
12.2 Step 1b — self-interference, with the bench kit on the aircraft . . . 22
12.3 Step 2 — the board on the bench, after T3 . . . 23
12.4 Step 3 — in flight . . . 24
12.5 TLEs and time . . . 24

13 Part 12 — T3c: IMU temperature calibration **24**

14 Part 13 — T3d: the pads and test points **25**

15 Part 14 — T4/T5: tethered, then flight, strictly in this order **25**
15.1 Foam, shroud and coating, after the bench and before the first flight . . . 25
15.2 Staking the plugs . . . 26
15.3 Order of flights . . . 26
15.4 T5b — the velocity the aircraft really has . . . 26

16 Part 15 — Audit: what is still manual, soldered, or unmeasured **27**
16.1 15.1 Manual soldering . . . 27
16.2 15.2 Manual assembly . . . 27
16.3 15.3 Measurements that have not been confirmed . . . 28
16.4 15.4 The three worst, ranked . . . 29
16.5 15.5 Design-level inefficiencies, for a future revision . . . 29

17 Part 16 — What only the outside world can settle **29**

18 Part 17 — What is not built yet **30**

19 The one-page order **31**

1 How to use this, and how to verify it

This is one document in the order you actually do things: order the board, order the aircraft parts, assemble, power it up, and fly it. It exists because the same information used to be spread across four draft documents (ORDER, ASSEMBLY, BUILD and BUYING) that had drifted apart — in three cases far enough to send you shopping for parts with nowhere to put them. On 2026-09-19 the first three were folded into this one and deleted; the buy list in Part 2 is generated from `docs/PARTS.csv`, the decision record.

Verify before you spend. Read the verdict, not a summary:

`python3 tools/preflight.py` → must print **READY TO ORDER**

The verdict is *derived* from a manifest of gated prerequisites, so an unrun tool, a non-zero exit, a failed check or an unclassified manifest entry is a blocking failure. It cannot be typed by hand and it cannot shrink silently. What it proves is that the *design* is orderable. It cannot prove anything about money, stock, or JLCPCB's own upload preview — those are Sections 2.8 onward.

1.1 The four commands that matter

`python3 tools/preflight.py`	the gate; must say READY TO ORDER
`bash tools/make_order_bundle.sh`	regenerate `fab/` and zip the order
`bash tools/build_firmware.sh`	build `arducopter.apj`, pinned to Copter-4.7.0

1.2 Freshness

`fab/stock-snapshot.json` is re-fetched from `jlcpcb.com` and `tools/check_order.py stock` blocks the gate once it is older than **7 days**. It was last refreshed 2026-09-19 and resolves **56 of 56** non-DNP codes with cover for two boards. Everything else in `fab/` is regenerated by `tools/make_order_bundle.sh`, which refuses to build the bundle if the copper layer count is not 6.

2 Part 1 — Order the bare board and its assembly

2.1 Regenerate, do not trust

```
bash tools/make_order_bundle.sh
```

This re-exports the gerbers and drill, re-runs `gen_bom.py` for *all three* variant pairs, checks the layer count, and writes `fab/NAVCORE-SoOP-order-bundle.zip`.

Do not hand-copy or hand-edit anything in `fab/`. A `cp` of a stale variant into the bundle is exactly how the Economic BOM once shipped quoting `Y2`'s superseded, unbuyable part code for three days — and the order notes of the day were recommending those files at the checkout while it did.

2.2 What to upload

File	Where
`fab/gerbers/` — zip the *contents*	PCB gerbers + Excellon drill
`fab/BOM-NAVCORE-SoOP.csv`	assembly BOM, the default build
`fab/CPL-NAVCORE-SoOP.csv`	pick-and-place, the default build

Or upload the bundle zip, which contains those plus `HOW-TO-ORDER.txt`. `fab/gerbers/` holds **16 files**: 14 `.gbr` (6 copper, 2 paste, 2 silkscreen, 2 mask, edge cuts, drill map), `NAVCORE-SoOP.drl`, and `NAVCORE-SoOP-job.gbrjob`.

> **Stop-condition at upload.** If the preview shows **four** copper layers, abort. The power planes are missing and the board will not work. It must show six.

2.3 Fabrication settings

Setting	Value	Why
Layers	**6**	In1 is a solid GND plane; In4 carries the rails
Dimensions	46.5 × 47.2 mm	the outline in `design.BOARD`
Thickness	1.6 mm	30.5 mm stack standard
Outer copper	**1 oz**	every current figure assumes it
Inner copper	**1 oz**	the VBAT plane on In4 needs it (`irdrop.py check`)
Surface finish	ENIG	0201s, LGA-14s and a DSBGA want a flat finish
Min track / spacing	**0.1016 mm (4 mil)**	should be free tier
Min via / drill	**0.45 mm / 0.20 mm**	
Impedance control	No	USB is a plain pair; the RF run follows the antenna's 28 dB
Via covering	**Epoxy Filled & Capped**	some vias sit in SMD pads
Stencil	not needed	everything machine-placed but Section 7
Mark on PCB	**Remove Mark**	no JLC marker on the board; small fee

The 1 oz line is not cosmetic: **every current-capacity figure in `check_mech.py build` assumes it**, and 0.5 oz halves each trace's rating.

2.4 Quantity, and the tier decision

Order 5 bare boards and 2 assembled. Five is the multilayer minimum; two is the usual SMT assembly minimum.

	Standard (panelised)	Economic
Board minimum	**70 × 70 mm**, so 2 × 2 panel + rails	10 × 10 mm, no rails
Places	everything, including `U3`	everything except `U3`
Feeder fees	included	**GBP 40–46** still charged
Hand-fitting	none	`U3` (ICM-42605, second IMU)

Recommendation: Standard, panelised 2 × 2, "panel by JLCPCB". Do not commit a panelised board file — the board files stay exactly as gated, so nothing has to go through DRC, connectivity and the whole gate again. A hand-panelised `.kicad_pcb` would have to, for no benefit.

If you take Economic instead, **take its BOM and CPL as a pair** (`*-economic.csv` for both). Mixing an Economic BOM with the default CPL hands JLC a placement list for parts the BOM does not order. `tools/check_order.py variants` — now a gated check — fails if any pair disagrees with the board.

2.5 The variant files, and why two of them are identical

Variant	BOM lines	Placements	Difference from default
`NAVCORE-SoOP` (default)	82	**180**	—
`NAVCORE-SoOP-economic`	82	**179**	omits `U3` only

There is one build and an Economic variant; a VTX runs from `+5V_PAYLOAD` on `J22`. Choosing Economic only makes sense if you want to hand-fit the second IMU.

2.6 Parts worth checking before you pay

Stock is no longer quoted in prose. It lives in the dated snapshot `fab/stock-snapshot.json`, age-gated at 7 days by `tools/check_order.py stock`.

Two substitutions are **not optional**:

- `Y1` **must be** `C2682774` — a *passive crystal*, CL 20 pF. The visually identical `C2901556` is an **active oscillator**: same SMD3225-4P land pattern, but pin 4 is VDD, which this board grounds. Substituting it gives a board that never starts.
- `Y2` **must be** `C19674258` — the ±0.5 ppm grade of the same HCI TCXO family. The ±2 ppm `C5563878` it replaces has the same pins and land, so a substitute fits, but it lets the tuner's reference wander four times as far.

2.7 The DNP list — derived, never typed

One designator ships unpopulated: `R15`, the 120 Ω CAN bus terminator. A CAN bus is terminated at its two *ends*; fit `R15` (0402, across `CANH` and `CANL`) only if this board is the last node on the bus. Revision D fits everything else, including the CAN transceiver `U11` (on its own `+3V3_CAN` LDO, `U21`, fed from the payload rail) and the payload buck `U20`.

Do not type this list — `make_order_bundle.sh` derives it from the BOM into `HOW-TO-ORDER.txt`.

2.8 Check these on screen at upload

- Outline and board size: **46.5 × 47.2 mm**
- Copper layer count: **6**
- Drill count and the 0.20 mm minimum
- **The placement preview: nothing should need turning.** Every rotation and centre in the CPL is fitted to JLCPCB's own footprint for that LCSC part (`tools/jlc_orientation.py`, 160 of 163 agreeing with the rule-based rotation it replaced). A part that still looks turned is a finding — report it rather than rotating it by hand
- **1 oz outer and 1 oz inner copper**, explicitly
- **Panel by JLCPCB.** Standard assembly requires a single PCB of at least 70 × 70 mm and this board is 46.5 × 47.2 mm, so the panel and its breakaway rails have to come from somewhere. Their own panelisation is the low-risk answer: you then confirm the panel outline in the same preview as the CPL.

2.9 Paying

- **Coupons are once-per-order** and cannot be combined. Take the SMT one.
- **6-layer promotions recur**; worth waiting a few days if you are not in a hurry.
- **UK import VAT is 20%**, collected by the courier above GBP 135 with an č8–12 handling fee. It is not a JLCPCB charge and is easy to forget when comparing against a UK-stocked alternative.

3 Part 2 — Order the parts: the board's bench first, then Iridium, then the aircraft

The list is staged by what you are trying to prove, not by what the aircraft needs. **A1 and A2 need no aircraft at all**: they power, flash and bench the board, and put an Iridium receive chain on the bench and then on the board. **B1 and B2 are the aircraft.** Order everything at once if you can — lead times run 2–4 weeks — but if the budget is staged, this is the order. The table is generated from `docs/PARTS.csv` (`python3 tools/gen_doc_tables.py`); each row's provenance and price tag are in the CSV.

A1 — The board, and its bench

10 lines, about GBP 345. Power it, flash it, prove it - no aircraft. Runbook Parts 7-10 (T1-T3a).

Item	Qty	GBP	Source
NAVCORE-SoOP PCB + assembly	1 order	190	JLCPCB
ST-Link V2 programmer	1	4	AliExpress
microSD card, 32GB	1	8	AliExpress / anywhere
USB-C cable, data-capable	1	3	anywhere
Spare MAX2112ETI+T tuners (U13), x5	5	75	DigiKey
JST GH 8P single-ended pigtail, GHR-08V-S (T1 bench feed into J2)	2	3	AliExpress
Bench power supply, adjustable with CURRENT LIMIT	1	35	AliExpress
Thermocouple meter or IR thermometer	1	14	AliExpress
Foam: open-cell block over the baro, closed-cell pad on the ESC	1 small sheet each	5	anywhere
Kapton tape, RTV silicone, and a foam cap for Y2	1 each	8	anywhere

Already have: Soldering iron and basic tools, Multimeter with continuity buzzer.

A2 — Iridium navigation, no aircraft

4 lines, about GBP 24. The receive chain the board cannot supply, and the bench reference receiver for T3b step 1. The tuner (U13) is on the board; this is what feeds it.

Item	Qty	GBP	Source
MMCX to SMA pigtail, 15 cm, RG178 or RG316, right-angle SMA (J12 to the HC610)	2	6	AliExpress
SMA DC block, inline, 10 MHz-6 GHz (bench)	1	10	AliExpress
SMA bias tee, 10 MHz-6 GHz (bench, powers the HC610 from the Pluto side)	1	8	AliExpress
gr-iridium + iridium-toolkit (software)	1	0	GitHub (muccc)

Already have: Raspberry Pi 5, Tallysman HC610 active Iridium helical antenna, ADALM-PLUTO SDR (bench reference receiver).

B1 — The aircraft - minimum to fly on GPS

28 lines, about GBP 389. Frame, propulsion, radio, GPS, fasteners, looms. Every optional sensor ships disabled, so nothing in B2 is needed for a GPS flight.

Item	Qty	GBP	Source
TBS Source One V5 7in DC	1	36	HobbyRC UK
Motors 2806.5 1300KV	4	60	AliExpress
Props 7040 or 7042	4 sets	14	AliExpress
SpeedyBee BLS 60A 4-in-1 ESC	1	48	AliExpress

Item	Qty	GBP	Source
RadioMaster Pocket ELRS transmitter	1	68	AliExpress
ELRS 2.4GHz receiver	1	15	AliExpress
M10 GNSS + QMC5883L compass (antenna INTEGRATED)	1	24	AliExpress
5V PASSIVE piezo buzzer	1	3	AliExpress
WS2812B LED strip	1	6	AliExpress
XT60 pigtails + silicone wire + heatshrink	1 kit	12	AliExpress
Smoke stopper (XT60)	1	10	AliExpress
EC5 to XT60 adapter	2	6	AliExpress
Landing gear - TPU skids (3D PRINT)	1 set	0	3D print
Landing gear - TPU/nylon skids, BOUGHT (backup for the printed set)	1 set	8	AliExpress
M3x14 socket screws (motor)	16	3	AliExpress
M3 NYLON female-female standoffs 15 mm (ESC-to-FC spacer)	4	2	AliExpress
M3x35 socket screws (stack bolts)	4	2	AliExpress
M3 aluminium standoffs, 25 mm x4 and 33 mm x4 (top plate)	8	4	AliExpress
M3 nylon washers (under the stack bolts' heads)	1 pack	2	AliExpress
M3 socket screw assortment (10/12/14/16/20 mm) + M3 washers	1 kit	5	AliExpress
M3 silicone grommets, spare set (30.5 mm stack)	1 pack	2	AliExpress
Heat-set inserts, CNC Kitchen M2 x 3 / M2.5 x 4	1 small pack of each	12	CNC Kitchen store
Brass screws M2x8 (beside the GPS)	1 small set	6	AliExpress
M2 and M2.5 socket screws + M2 nylon lock nuts	1 kit	5	AliExpress
JST GH 4P pigtails, GHR-04V-S (the sensor, RC and CAN leads)	1 pack of 10	5	AliExpress
ESC lead, JST GH 8P to SH 8P, straight-through (J2 to the ESC)	2	6	AliExpress
JST GH 6P leads, GHR-06V-S (J21 companion; spares for J3 GPS and J14 flow)	3	5	AliExpress
TPU 95A filament 2.85 mm, for the skids	1 spool (250-500 g)	20	anywhere

Already have: 4S LiPo packs, LiPo balance charger.

B2 — Aircraft sensors and payload - optional

5 lines, about GBP 68. RNGFNDn_TYPE and PRX1_TYPE ship at 0; enable each one at the bench after it is fitted. The +5 V rail cannot carry all of these at once - check_order.py modules prints the budget.

Item	Qty	GBP	Source
Benewake TFS20-L lidar	1	32	AliExpress (or 3DXR UK GBP35.70)
VL53L1X ToF module (GY-53-L1X)	1	6	AliExpress
LDROBOT LD06 360 lidar (INDOOR USE ONLY)	1	14	eBay - Okdo Lidar Hat kit (use the LIDAR UNIT only)
XIAO ESP32S3 Sense (recording camera)	1	14	AliExpress
JST GH 3P lead, GHR-03V-S pigtail (J22 payload 5 V)	2	2	AliExpress

Total, everything to buy: about GBP 826. Generated from `docs/PARTS.csv` by `tools/gen_doc_tables.py`; every row's reasoning, link and provenance is in the CSV.

Do not buy a bare PMW3901 or VL53L1X chip to hand-fit. Both were deleted from the board in the Rev B re-layout and have no pads; `docs/BUYING.md` priced a "GBP 227 hand-fit route" around them until 2026-09-19. The off-board GY-53-L1X *module* for the upward rangefinder is a separate, later purchase and is unaffected.

3.1 The one order — nothing here should make you order twice

Every "measure this, then use a different part" branch in this document is closed by buying the alternatives up front. The premium is about **GBP 6**.

What to add	What it closes
M3 socket assortment + washers	Both derived joints round *up* past their engagement target (4.5 mm and 8.0 mm into the thread) and the skid's tapped depth is the only measurement that can invalidate the length you picked.
4 × 15 mm female-female spacers	the ESC-to-FC separation is 13.5 mm and `fasteners.py` says plainly that a grommet cannot hold it; 15 mm is the next length sold. Without these the stack cannot be assembled at all.
25 and 33 mm top-plate standoffs	replace the kit's 22 and 30 mm: the JST-GH connectors under the board raised the stack, and with the kit's the top plate touches it at the loose-grommet corner. Do *not* buy 35 mm standoffs for the stack pattern; the top plate has no 30.5 mm holes.
spare **JST-GH 6-pin** lead	the residual `J3` risk is a *bulky moulded boot* on the lead you receive, not the board. Test-fit the plug on arrival; a second lead is then the fallback rather than an interruption.
10 pre-crimped JST-GH 4P pigtails, a GH 3P and the GH 6P leads	four ports (`J5`, `J6`, `J9`, `J11`) take a GH 4P, `J22` a GH 3P and `J21` a GH 6P; a spare of each means a bad solder joint is not a project stall
the GH 8P to SH 8P ESC lead, two	`J2` is JST-GH and the ESC's socket JST-SH, so the leads in the ESC's box do not fit; without this lead nothing flies
the HC610 antenna, its MMCX to SMA pigtail, the DC block and the Blog V4 dongle, all with the first order	T3b step 1 needs the antenna and the dongle; the aircraft needs the antenna and the pigtail; the DC block protects any bench instrument from the +5 V on `J12`

Three things need the parts in hand: the frame kit's top-plate standoff spacing (the kit supplies it), the camera mount (deferred, and the XIAO is the camera it must take), and the Radxa Zero 3W's hole pattern (deferred *with the companion* — the vendor never published it, so the tray is a print against a measured board). The Radxa and the flow camera are deferred *purchases*; when they come, the Radxa talks over `J21` and takes its own BEC. The XIAO records to its own microSD and takes only 5 V from `J22`, so it is ordered now.

3.2 The bench measurements that could have cost you a purchase

Sort the bench items by what a result could move, because only one kind can reach your wallet. `python3 tools/check_order.py bench_bounds` does this sort now and the gate walks its verdict, so the answer is printed rather than argued. Nine items became six on 2026-09-20, in five kinds:

- **CLOSED** — the answer was already fixed by a *document* or by the artwork, so there is nothing left to measure that could change the board. Three items left the list this way: the grommet, `BATT_AMP_PERVLT` and the `J2` pin order. They are not deleted — `check_order.py bench_bounds` still prints each with its evidence, and a reconciliation step refuses if this list and the manifest ever disagree.
- **BOUND** — the design-side corner is already under the limit and the measurement *confirms* it. `U8`: 87 °C against 125 °C. `U9`: 104 °C on good copper against 150 °C, on a board `thermal_vias.py` measures at 7443 mm^2 of plane. The grommet: the *metal* M3 head clears copper at every hole by +0.08 mm, and the flange is always the wider of the two, so a wider grommet adds only silicone bearing on mask.

- **PARAM** — the result is a *firmware constant*: `FLOW_ORIENT_YAW`. The board carries the pin and has no value to change.
- **MODULE** — the property belongs to a *bought part* that plugs in: the RF front end, which is the Tallysman HC610 active antenna, and flow quality, which is the off-board camera. A shortfall is placement, shielding or a different module.
- **DATA** — the T3b record asserts nothing about the board at all.

The consequence is the one that matters before you pay: **no bench result can force a different board**. Every item is still measured, but it confirms a bound instead of discovering one, so the order does not wait for the session. The old text here warned that a hot `U8` meant a respin; that risk was closed on 2026-09-20 by the part swap (Part 10), which moved the worst corner to 87 °C and left it on the board.

Three are now closed outright rather than bought around. Each names the evidence:

Item	What fixed the answer, so nothing is left to measure
the grommet flange	the flange and the M3 cap head are *concentric*, so the flange is always the wider of the two and the metal always the narrower. A wider grommet adds only silicone bearing on mask; it cannot make the metal reach copper. Z was already swept 2.0–4.5 mm by `check_mech.py fit`, which requires every purchase to cover the range, so the design is *invariant* over the grommet
`BATT_AMP_PERVLT`	the ESC's own manual publishes Scale=400, i.e. 40 mV/A, i.e. 25.0 A/V — and `defaults.parm` ships exactly that. The value was typed in two places, so `gen_hwdef.py` now *derives* it from `design.ESC` and `check_mech.py build` asserts it against the shipped file. Mutation-tested with the retired 52.7
the `J2` pin order	`design.ESC` now *declares* the manual's order and `check_mech.py build` holds the netlist against it. Nothing checked this before: `J2` is a passive connector, so DRC, ERC and the pin-semantics check all pass a transposed one. The symptom is four motors on the wrong signals

The honest residue: no bench item can change what you *order*, and that is now a gated claim rather than a reading of this section — `check_order.py bench_bounds` fails if any item is unclassified, if a bench entry is added without one, or if this list and the manifest disagree. *One* item has no analytic bound and stays a flight gate: whether the RF link closes depends on real sky and real desense, which no desk calculation settles. The one risk that *was* real has been closed: the old `U8` corner of 139 °C was a budget error, and on 2026-09-20 (Part 10) the part was swapped in place to the LMR33630ARNXR, taking the worst corner to 87 °C against 125 °C. **Three spare bare PCBs** stay on the order for misadventure, not for a known unknown. `tools/fasteners.py` prints this list itself under *"not derived — these need the parts in hand"*.

4 Part 3 — While you wait (free, and the highest-value work)

1. **Register ArduPilot board ID 7180.** Not needed to order; needed before sharing. The request is written out in `firmware/BOARD-ID-REQUEST.md`, with the registry check, the alternates and the issue text. **Do not request 9001** — it is free but it is above the #7199 ceiling that `board_types.txt` explicitly asks you not to cross, while 4544 free IDs sit below it.
2. **Read the two screw warnings in Section 6.2 twice.** They are the ones that strip a frame.
3. **Print `docs/img/padmap.svg` and the assembly drawings `fab/assembly-top.svg` and `fab/assembly-bottom.s`** Every reference designator is on the assembly drawings, and Section 6.5 has every connector by

edge. Pads print their function (`L5V`, `DIN`, `BZ` ...) rather than a designator, and only where it sits unambiguously beside its pad (Section 14).

4. **Decide the board's orientation before cutting any loom.** All four edges carry connectors, top and bottom; the aft edge (`J3` GPS, `J12` antenna) faces the antenna tower. Read the orientation flag in Section 5.5 first.
5. **Dry-run the SITL suite** if you have a Linux box: `bash sitl/run_scenarios.sh`.

5 Part 4 — On arrival: inspect before you spend more

5.1 Visual inspection

Under magnification, look at `U1` (**LQFP-100, 0.50 mm pitch**), `U2`/`U3` (**LGA-14**, no accessible leads), and `U13` (**TQFN-28 with an exposed pad**). You are looking for bridges and tombstones. JLC is good at this. Look anyway.

5.2 Confirm the machine-readable facts

`python3 tools/preflight.py` never mind the board — confirm DRC, ERC, gerbers and BOM/CPL are all current

`python3 tools/check_order.py variants` every variant pair matches the board

5.3 The frame measurements that settle a real decision

Check with calipers	Parsed value
top plate width	**42.50 mm** — the battery is 47 mm and already overhangs 2.25 mm each side; there is no room for an upward ToF beside it
bottom plate M3 positions	x = ±14.60 and ±11.00, **no 30.5 mm pattern** — so the FC cannot bolt under the bottom plate; that idea is closed
which plate is top	**inferred** from length (160.26 mm). The one genuinely open row

5.4 The two things no drawing can answer

- **grommet compression under load** — a property of the rubber
- **the real TPU gear's shape, hole pattern and strength** — the assembly is modelled as four legs at the motor pattern on assumed dimensions

5.5 T0 — the mechanical checklist, before any loom is cut

Every figure here is recomputed by `python3 tools/check_mech.py mechanical`; if the board has moved since this was compiled, run it rather than trust the table.

- **Centre-plate opening** clears 46.5 × 47.2 mm plus the plugs on every edge. The FC plate is 48.50 mm across the binding axis, so the board has only **0.55 mm** on the tight side in the worst-case orientation (`python3 tools/check_mech.py mechanical` prints it). Check it with the real plate.
- **Orientation.** The aft edge carries `J3` (GPS) and `J12` (antenna) and faces the tower; the fore edge carries the servo headers. The silkscreen arrow points at the fore edge, and the hwdef's IMU rotations are derived with that edge forward (`BOARD_FORWARD` in `tools/design.py`), so mounted this way `AHRS_ORIENTATION` stays 0. Mounted any other way, set it to match. Confirm it on the bench at T3.1 (roll and pitch it, watch the horizon) before any flight.

- `J1` **USB-C** is on the east edge and needs about **9 mm** clear in front of it for the plug's overmould. A standoff, a plate lip or the battery strap across that edge makes the port unusable, and it is the only way to flash the board.
- **The servo plugs** on `J17`/`J23` sit on pins that overhang the fore edge and need about **17 mm** clear for the plug and the lead's bend (`design.MATING_CLEARANCE`); the printed clip sits over them.
- **Every JST-GH port** needs about 6 mm in front of its mouth, top and bottom. `J2` (ESC) and `J22` (payload) are side entry on the west edge, so their plugs go in horizontally.
- **microSD** is on the underside of the west edge and needs about 14 mm to withdraw the card. Nothing on the frame stands in that path, so the card comes out with the board in the stack, provided the `J2` and `J22` leads loop about 20 mm out from the edge before they drop, as `tools/cables.py` routes them. Dressed tight to the edge, they hang across the card's way out.
- **Grommets** must hold 3.0 mm between the ESC's parts and this board's under compression; the four 15 mm nylon female-female spacers set the ESC-to-FC separation. The compressed height of the rubber is the one *assumed* figure in the stack.
- **Nothing metal wider than a cap head at any hole.** The nearest copper to a hole centre reads 3.32 mm (`TP9`) on `tools/dimensions.py`: an M3 cap head (2.75 mm radius) clears by 0.57 mm and a 7 mm washer overlaps by 0.18 mm, so the washers under the heads are nylon (`design.FC_HOLE_HARDWARE`).
- **Skids match the 19 × 19 mm motor pattern**, not the frame's 16 × 16.

6 Part 5 — Mechanical assembly

6.1 Stack height

31.6 mm against the top plate's underside at 35.5 mm — 3.9 mm spare. The top plate rides on **25 mm front standoffs**, which stand on the mid plate (bottom plate 2.5 + arms 6.0 + mid plate 2.0 = 10.5 mm, + 25 = 35.5 mm); the kit's 22 mm ones, now retired, left 2.2 mm before the JST-GH connectors under the board added 1.35 mm, and none at the loose-grommet corner. The 31.6 mm is `check_mech.py mechanical`'s figure and includes `J3`'s mated plug; `check_mech.py build`'s 19.8 mm is bare parts only and is not the one the plate must clear. Earlier drafts said "22.3 mm against 25 mm, 2.7 mm spare" — a Mark4-era number, retired. There is *less* room than that, not more.

6.2 The two screw warnings — read both

```
python3 tools/fasteners.py
```

- **Motor to arm, skid sandwiched.** Needs 13.5 mm, so order M3 × 14. But rounding 13.5 → 14 drives **5.5 mm into a thread designed for 4.0 mm**. A motor boss is often only about 4 mm deep, and **a bottomed-out screw feels tight while clamping nothing** — that is the failure that throws a motor in flight. Measure the tapped depth before fitting sixteen screws. M3 × 12 plus a washer is the usual fix.
- **FC to ESC to frame, the 30.5 mm stack bolt.** Needs **31.0 mm**, so M3 × 35. Rounding 31.0 → 35 puts **8.0 mm** into the press nut. If the tapped depth is shallower than that the screw bottoms out and clamps nothing; an M3 × 33 plus a washer is the fix.

This is the whole reason the buy list carries an M3 assortment rather than one length. `free_island` is not involved; the point is that both joints are decided by a measurement you have not taken yet, so own every plausible length.

6.3 Screw heads against copper

No copper is under any screw head. The closest is `TP9` at **3.32 mm** from a hole centre against a **2.75 mm** cap-head radius — 0.57 mm — and the washer under each head is nylon, so even a 7 mm

washer that reaches the pad puts no metal on copper. If the board has moved since this was compiled, `python3 tools/dimensions.py -md` prints the current figure.

6.4 Sequence

1. **Inspect the boards** (Section 4.1).
2. **Print the assembly drawings and the pad map** (Section 14).
3. **Dry-fit the FC into the centre plate.** Confirm nothing blocks the west edge's underside, where the microSD ejects.
4. **Buzz the GH 8P to SH 8P ESC lead**, all eight pins end to end, against the `J2` pinout below, before anything is powered.
5. **Sit a skid under each arm**, holes aligned. Motor on top, wires inboard. Thread all four screws by hand before tightening any; tighten diagonally.
6. **Solder the ESC's capacitor first.** The ESC comes with a low-ESR electrolytic capacitor in the box. Solder it across the ESC's battery pads, observing the polarity marking, with the legs as short as possible. It soaks up the spike when the battery is plugged in and the ESC's own switching spikes. Without it the board still survives a 5S plug-in (32.5 V peak in simulation against the regulators' 38 V limit), but the margin shrinks to 5.5 V. If the box has none, buy a 35 V, 470–1000 µF low-ESR part.
7. **The stack, bottom to top: ESC, grommets, FC.** Fit the silicone grommets into the FC's four mounting holes; ESC into the frame on its 30.5 mm pattern; FC on top with its aft edge (`J3`, `J12`) toward the antenna tower and the silkscreen arrow pointing at the nose; `AHRS_ORIENTATION` stays 0 (Section 5.5).
8. **4 × M3 × 35 through the stack**, with the four 15 mm female-female spacers setting the ESC-to-FC separation.
9. **The top plate on the bought standoffs** — 25 mm at the front, over the stack, 33 mm at the rear, in place of the kit's 22 and 30. Both sets change together or the top plate is not flat.

6.5 What plugs in where

Every cable port is a latching JST-GH 1.25 mm connector except the two servo headers and the antenna. Pin 1 first; "underside" is the board's bottom face. From `design.FLOORPLAN_REV_D` and the netlist.

Port	Part	Pins, and what plugs in
Aft edge — toward the antenna tower		
`J3` GPS, top	GH 6P	1 +5V, 2 `USART2_TX`, 3 `USART2_RX`, 4 `I2C1_SCL`, 5 `I2C1_SDA`, 6 GND. The GPS module's own lead, latched
`J12` antenna, top, NE corner	MMCX, right angle	the HC610's pigtail. **Carries +5 V DC** whenever the feed is on (the default)
`J9` I2C, underside	GH 4P	1 +5V, 2 SCL, 3 SDA, 4 GND. The rangefinders, split at a junction
`J6` DroneCAN, underside	GH 4P	1 `+5V_PAYLOAD`, 2 CANH, 3 CANL, 4 GND. A DroneCAN node's own GH lead mates
Fore edge		
`J17` SERVO1, top	1×3 2.54 mm male, SMD right angle	1 `PWM7`, 2 `+5V_PAYLOAD`, 3 GND — the standard servo order

Port	Part	Pins, and what plugs in
J23 SERVO2, top	same	1 PWM8, 2 +5V_PAYLOAD, 3 GND
J5 RC, underside	GH 4P	1 +5V, 2 USART6_TX, 3 RC_IN, 4 GND. The ELRS receiver
J11 lidar, underside	GH 4P	1 +5V_PAYLOAD, 2 USART1_TX, 3 USART1_RX, 4 GND. The LD06 (later)
East edge		
J21 COMPANION, top	GH 6P	UART7 = SERIAL1, Pixhawk TELEM order: 1 +5V, 2 TX, 3 RX, 4 CTS, 5 RTS, 6 GND
J1 USB-C, top	USB-C	setup and flashing; unplugged before flight
J14 flow, underside	GH 6P	SPI3: 1 +3V3, 2 SCK, 3 MISO, 4 MOSI, 5 CS, 6 GND
West edge		
J2 ESC, top	GH 8P, side entry	1 GND, 2 VBAT, 3–6 M1–M4, 7 CUR, 8 TEL — SpeedyBee's order
J22 PAYLOAD, top	GH 3P, side entry	1 +5V_PAYLOAD, 2 GND, 3 GND. A 5 V VTX or the recording camera
J8 microSD, underside	TF-01A	the card: logs and Lua scripts
Soldered pads		
P151–P153, SE corner	pads	+5V_PAYLOAD, WS2812 data, GND — the LED strip
P161–P163, SW corner	pads	+5V, buzzer −, GND — the passive buzzer across P161/P162
Bench port		
J19 SWD	Tag-Connect TC2030-NL	1 +3V3, 2 SWDIO, 3 NRST, 4 SWCLK, 5–6 GND; fallback only (Part 8)

Serial leads cross over. The board's TX goes to the module's RX and the board's RX to the module's TX: on J5, pin 2 (TX) to the receiver's RX and pin 3 (RC_IN) to its TX. A Pixhawk GPS lead on J3 is already crossed.

J21, the companion port — three rules.

1. **Pin 1 is +5 V for a radio-class load only** — a SiK radio, an ESP32 MAVLink bridge. A companion computer (a Radxa wants 5 V at 2 A) takes its own BEC and uses pins 2–6, with its ground joined to pin 6.
2. **It is a Pixhawk TELEM port**, so a lead made for one mates; check that the far end crosses TX and RX. CTS and RTS are on pins 4 and 5; set BRD_SER1_RTSCTS to match what the far end wires, and leave it 0 if only TX/RX/GND are connected.
3. **It is the only spare serial port.** SERIAL6 is unassigned (UART4's pins became the PWM5 and PWM7 outputs), and J11 is earmarked for the lidar. A second serial device means giving one up.

J12 carries +5 V. The antenna feed (U24, current-limited to 95–155 mA; a dead short folds back to about 35 mA) is on from power-up, because PC5 is pulled up and the firmware keeps it on. That powers the HC610's LNA up the coax — and it will push DC into anything else you connect. **A bench SDR, a signal generator or a spectrum analyser on J12 or its pigtail goes through the DC block**, or put ant_feed 0 in APM/SOOP/soop.cfg and reboot first. Never short the centre pin to test it.

The ESC lead is GH 8P at the board and SH 8P at the ESC. J2 is wired in the SpeedyBee's own order, so the lead is *straight-through*: pin 1 to pin 1, through pin 8. The SH-to-SH leads in the ESC's box do not fit J2. **Buzz the lead you buy or splice, all eight pins, before any VBAT**:

pin 2 is the battery, and a lead with two pins swapped puts VBAT on a motor signal or ground.

Servos on J17 and J23. Each is a male 1×3 header in the standard order, so a servo's own female plug goes straight on: signal (orange, yellow or white) on pin 1, red in the middle, brown or black on pin 3 — check pin 1 on the assembly drawing before pushing the plug home, because a reversed plug fits. The pins overhang the fore edge; **fit the printed clip over both plugs** so a pulled lead loads the clip and not the solder joints. For two 9 g micro servos the header's own pin 2 feeds them from +5V_PAYLOAD. Anything bigger goes on its own 5–6 V BEC, with the BEC's ground joined to pin 3 and pin 2 left unconnected. lander.parm already drives SERVO7/SERVO8; check the travel direction with the props off.

Mount the compass as far from the ESC and the battery leads as the frame allows. There is no magnetometer on the FC by design: one 15 mm from a 60 A 4-in-1 is useless.

The payload rail is budgeted per mission. U20 and its inductor L5 carry 2.1 A continuous. The drone's payload (VTX or camera on J22, LED, LD06, CAN) is about 1.0 A; the lander's (two TVC servos, LED, CAN) is 0.76 A. **Wire one mission's payload, not all of it**: every payload connector at once is over what U20 should carry continuously, and check_order.py modules fails any declared mission that does not fit. The Iridium antenna is not on this rail: its feed comes off the flight controller's +5 V through U24.

6.6 Attaching the Iridium antenna

One cable: the MMCX to SMA pigtail (15 cm, RG178 or RG316, straight MMCX, right-angle SMA).

1. **Fit the HC610 to the tower seat** with its two M2.5 × 8 screws into its own threads (6 mm deep — never longer than 10), its SMA male pointing down the column.
2. **Mate the pigtail's SMA female onto the HC610's SMA** inside the column, finger tight and then a small turn with a wrench. The right-angle end turns the cable straight out through the column's starboard window.
3. **Tie the coax round the column** just below the window, so the snap and the SMA carry no strain. Keep the first bend at least 10 mm from each connector.
4. **Route it forward outboard of the battery** and in past the top plate's starboard edge to the board's aft edge (the route tools/check_mech.py cables checks), and snap the MMCX plug onto J12 until it clicks. Tie it down 10–15 mm from the plug with a little slack, so vibration works on the tie and not the snap.

First power with the antenna on. Check these at T2, once the firmware is on:

- The GCS shows *SoOP: antenna ok, 15.0 mA* — about 15 mA is the HC610's LNA. *SoOP: antenna open* (under 3 mA) means the pigtail is not mated or the antenna is dead; *SoOP: antenna short* (28 mA or more, or the switch's fault flag) means a short on the coax — the firmware turns a short that lasts 5 s off for 10 s and retries. The log records it once a second as SOA.
- The tuner locks: *SoOP: tuner locked, VCO n* at boot (Part 11, step 2).
- The orbits on the card are fresh: APM/SOOP/iridium.tle fetched within the last few days. The filter ignores elements older than 7 days (Part 11, TLEs and time).

6.7 The downward radar — research first, then this

The radar is the velocity source the GNSS-denied fix is waiting for. In simulation the error is set by the flight EKF's velocity once the GPS is gone: 220 m median at 1 m/s, 33 m at 0.1 m/s, 12 m at 0.03 m/s (python3 tools/check_soop.py sweep). Better orbits, a steadier TCXO or a better barometer barely move it. A TI IWR6843AOP looking down reads each ground return's speed from its Doppler, so grass and sunlight do not defeat it the way they defeat a flow sensor, and a GPS spoofer cannot drag it. It is not an obstacle sensor: its ±60° cone looks at the ground, never at the horizon.

Nothing in the firmware reads it yet. Buy it after the desk research (the module, the chirp that sees ±20 m/s, the published ego-velocity error fed through the sweep), and fly it only once AP_SoOP has the driver and check_soop.py gates it.

1. **Measure the board** before printing. TI publishes no dimensions, so `design.MOUNTED['radar']` is assumed (30 × 22 × 9 mm, 15 g). Enter the real figures, run `python3 tools/gen_scad_mounts.py -stl` and `python3 tools/check_mech.py cad_fit`. The cradle has 26 mm between the plate's front edge and the lidar bracket; if the board is deeper than 22 mm, the radar moves aft and the TFS20-L forward.
2. **Check the nut side.** The cradle hangs from the plate's forward M2 pair, with nuts on the plate's top face under the front arms' roots. Look before printing: if there is no room for a nut, the radar takes the aft position instead.
3. **Caliper the LD06's window height** above its base and enter it as `OFFBOARD['lidar']['scan_plane_height_m`. Until then cad_fit warns that it cannot say whether the radar cuts the LD06's scan; the radar hangs 15.5 mm, the LD06 38.3.
4. **Break off the mission board** and tap the IWR6843's CLI UART RX and data UART TX on the chip side of the CP2105: the EVM's UART otherwise leaves as USB, which the H743 cannot host. Wire them to a GH-6P lead for `J21` (SERIAL1), its 5 V pin left unconnected. The radar draws about 2 W, so its 5 V comes from its own BEC off the battery leads: `J21`'s pin is the FC rail, and on the payload rail its 0.4 A would take `U20` to 132 °C on the drone mission. Fit TI's heat sink, which faces the cradle's vented floor.
5. **Fit it face down** in `radar_cradle` (two M2 × 8 through the counterbored floor into lock nuts on the plate), then `radar_lid` (two M2 × 4 into the cradle's inserts). The LD06 fills the aft 132–228° of its cone from 33° off nadir; `design.RADAR_VIEW` lists that wedge, and cad_fit fails a blocked direction it does not.

6.8 The harness — six runs, and how not to make a nest of them

Run	Wires	Note
`J2` → ESC	8	the GH 8P to SH 8P lead; stays inside the stack
`J3` → GPS	6	the module's lead, aft to the tower; the compass rides on it
`J12` → HC610	coax	the MMCX to SMA pigtail, aft to the tower, tied to its arm
`J9` → rangefinders	4	one lead, split at a junction: down and up
`J5` → ELRS	4	forward to the nose mount
`P151`–`P153`, `P161`–`P163`	3 + 2	the LED strip and the buzzer, soldered

Four routing rules, one of them specific to this aircraft:

1. **Keep every signal cable and the Iridium coax away from the four motor leads.** They carry kilowatts switched at tens of kHz. On a normal quad that is good practice; on this one it is the difference between hearing Iridium bursts and not (T3b).
2. Route along the arms and plate edges, zip-tied every 40–50 mm. Slack belongs in one service loop near the stack.
3. Leave the **USB port on the east edge** clear (Section 5.5).
4. The ELRS antennas and the Iridium antenna get separation from everything else, and from each other: ELRS at the nose, the Iridium helix on the tower at the tail.

6.9 Things that will bite you

- The pack is EC5; everything else is XT60. Use the adapter — do not re-terminate a hard-case pack on build day.
- The frame offers 16 × 16 and 19 × 19 motor patterns. The motors are 19 × 19.
- Measure both tapped depths before committing sixteen screws.
- The ESC's own leads do not fit `J2`; the GH 8P to SH 8P lead is a gamble until you buzz it.

- J12 carries +5 V: a bench instrument goes on through the DC block.
- A servo plug fits either way round on its header; signal is pin 1.
- Identify every pad by its label or on the padmap, never by counting from a neighbour.
- The board is **5S maximum** — the input TVS sets it (Part 1).
- Wire one mission's payload onto +5V_PAYLOAD, not every payload at once.
- Confirm 1 oz outer and inner copper on the order; 0.5 oz halves every trace's rating.

6.10 J2, the ESC connector — confirmed rather than assumed

Wired **1 GND, 2 VBAT, 3 M1, 4 M2, 5 M3, 6 M4, 7 CUR, 8 TEL** on a JST-GH 8P side-entry socket. Betaflight's board page for the SpeedyBee F405 V4 gives the same FC-side order on the SpeedyBee's JST-SH 8P, so the GH 8P to SH 8P lead is straight-through — no pin is moved.

The vendor table reads oddly because it lists the two *ends* of the cable in opposite order, since they face each other. Reverse the ESC column and every pair lines up: GND↔GND, BAT↔VBAT, M1↔S1, . . . , CUR↔CURRENT.

Buzz it anyway. One automated read of a wiki table is not the same as measuring the lead you have. A spliced lead is the easiest place to swap two wires.

> **Pin 8 is inert.** The ESC end is N/A — stock BLHeli_S has no telemetry output at all. SERIAL5_PROTOCOL 16 and the ESC_TEL wire are correct and cost nothing, but they will receive nothing until a telemetry-capable ESC (BLHeli_32, AM32) is fitted. That is why the board ships INS_HNTCH_MODE 1 (throttle) rather than 3 (RPM). The upgrade path needs no wiring: flash **BlueJay** or **AM32** onto the ESC over BLHeli passthrough (SERVO_BLH_AUTO 1 already enables it), set SERVO_BLH_BDMASK 15 for bidirectional DShot on motors 1–4, then INS_HNTCH_MODE 3 — the RPM-tracked notch is the single biggest flight-quality win available on this aircraft.

7 Part 6 — Soldering: the buzzer and the LED strip

Everything else plugs in. Identify every pad on docs/img/padmap.svg before the iron touches it — never by counting from a neighbour; three of these six pads print no label.

- **Buzzer** — to the pads in the SW corner: its + to P161 (+5 V), its − to P162 (the drain of Q1). P163 is a spare ground. Must be a **passive** piezo: PA15 is a timer channel and ArduPilot generates the tone patterns itself. An active buzzer has its own oscillator and loses every arming and failsafe pattern.
- **LED strip, optional** — to the pads in the SE corner: P151 (+5V_PAYLOAD), P152 (data, 5 V from U17), P153 (GND). 5 V WS2812B only.

8 Part 7 — T1: first power, before the MCU sees anything

USB disconnected. No battery. Do not skip this. A board that goes straight to USB can destroy the MCU on a rail fault that a current-limited supply would have caught for nothing.

Bench supply on VBAT, **16.8 V, current limit 100 mA**. VBAT_IN reaches only J2.2 — there are no separate battery pads — so feed it through the bare-ended GH 8P pigtail from the order (J2.1 GND, J2.2 VBAT, the other six insulated). Powering through the ESC instead works, but the ESC's own draw then sits inside the 80 mA below.

#	Check	Expect	If wrong
1.1	current at power-on	< **80 mA**	something is shorted: kill it and inspect U8, U20 and the bulk caps
1.2	+5V rail	**5.02 V** ±0.2	U8 LMR33630ARNXR, L2, divider R6/R7
1.3	+3V3 rail	3.3 V ±0.1	U9 TLV75733P
1.4	+3V3A rail	3.3 V ±0.1	U10 TLV75533
1.5	+5V_PAYLOAD rail (J22.1)	**5.02 V** ±0.2, on at power-on	U20 LMR33630ARNXR, L5, divider R42/R43; if dead, Q3 is holding U20's EN low — check R45
1.6	+3V3_CAN rail	3.3 V ±0.1	U21 XC6206, fed from +5V_PAYLOAD
1.7	antenna feed, J12 centre pin, nothing attached	about 5 V	U24 (enabled by R59's pull-up), R62, L6. Meter only — never short it

Raise the limit to 500 mA only once every rail is correct.

Then load both bucks. A regulator on the edge of stability reads a perfect 5.02 V at no load. Each buck has 2 × 22 µF on its output where TI's reference uses 4 × 22 µF, so check them under load before trusting them. Raise the limit to 2 A. Put a 5 Ω, 10 W resistor across J22 pins 1 and 2 (1 A from +5V_PAYLOAD, through a GH 3P pigtail) and a 10 Ω, 5 W one across J9 pins 1 and 4 (0.5 A from +5V, through a GH 4P pigtail). Both rails must stay 4.9–5.1 V, and L2, L5, U8 and U20 must not whine or get too hot to touch within a minute. With a scope, connecting the load should give one dip that settles within about 100 µs, not a ring or a steady oscillation. **If either rail rings or whines**, solder a 22 µF 16 V 1206 ceramic on top of each output cap (C22/C23 for +5V, C69/C70 for the payload rail) and test again. That is rework, not a respin.

> **If a rail is dead with a correct supply, look at R46 before the buck.** It is Q4's gate pull-down. Without it the gate floats, the FET never turns on, and the board is dead with everything wired correctly.

> **Reverse polarity is protected, but do not test it.** Q4 (WST4041 P-FET) blocks a reversed pack and DZ1 clamps the gate, but D1 is still a TVS rather than a series diode. Use the XT60 keying and check twice.

> **USB powers the logic, not the whole board.** D_USB feeds VBUS into the +5 V rail at about 4.7 V, which runs both 3.3 V rails, the MCU and the sensors — enough to configure and flash it. The payload rail stays off (its buck needs about 6.6 V), and through U8's internal diode USB also lifts VBAT, and via Q4 the ESC's battery pin, to about 4 V. Do the first power-up (this Part) on VBAT regardless.

9 Part 8 — T2: MCU alive

A blank STM32H743 already has a bootloader: the ROM DFU loader, which runs when BOOT0 is high through a reset. This board has the button for it (SW1 pulls BOOT0 to 3.3 V; SW2 is reset), so the first flash is over USB and needs no debug probe. Earlier drafts said "bootloader over SWD first, a board with no bootloader will not enumerate" — that was wrong for this MCU. SWD is the *fallback*.

1. Supply on VBAT (Section 8), as the board will be in flight; USB alone would also run the MCU. Data-capable USB-C cable — charge-only leads enumerate nothing.
2. **Hold BOOT (SW1), tap RESET (SW2), release BOOT.** `lsusb` shows `0483:df11 STM32 BOOTLOADER`. If it does not, check Y1 first (the passive crystal — Section 2), then fall back to SWD below.
3. **Flash the ArduPilot bootloader over DFU:** `dfu-util -a 0 -s 0x08000000:leave -D Tools/bootloaders/N` (built by `tools/build_firmware.sh` step 3). The board re-enumerates as the ArduPilot bootloader with the board's own ID.

4. Flash `arducopter.apj` from `tools/build_firmware.sh`, pinned to Copter-4.7.0, from a ground station or `Tools/scripts/uploader.py`.
5. Connect a ground station; confirm the board ID and read **every** boot message — every fitted sensor should probe, and nothing unfitted should be claimed.
6. **Copy `copter-deadreckon-home.lua` to the microSD at `APM/scripts/` by hand.** The build script does not package it into the `.apj`. Without it, `FS_EKF_ACTION 0` ships with no GPS-loss response at all.

SWD, if DFU never appears. `J19` is a Tag-Connect TC2030-NL pad set (1 +3V3 as the target-voltage reference, 2 SWDIO, 3 NRST, 4 SWCLK, 5–6 GND), reached with a TC2030-IDC-NL cable and an adapter to the ST-Link's header — neither is on the buy list, because DFU is the route. Flash with `st-flash write Tools/bootloaders/NAVCORE_SoOP_bl.bin 0x08000000` or STM32CubeProgrammer. It gets the same bootloader onto the same address; from step 4 on nothing differs.

10 Part 9 — T3: sensors on the bench, props OFF

#	Check	How	Note
3.1	IMU1 (`U2`) orientation	roll/pitch, watch the horizon	**worked out from the pads, never measured**
3.2	IMU2 (`U3`) orientation	same, `INS_USE2`	only if `U3` was fitted
3.3	Barometer	tracks a 1 m lift	
3.4	Battery voltage	against a meter	`BATT_VOLT_MULT 11.000`, should be close
3.5	**Battery current**	known load	ships `BATT_AMP_PERVLT 25.0` — a starting point, not a calibration
3.6	RC input	`RC_PROTOCOLS 512` (CRSF)	set to 1 for SBUS/FPort
3.7	Motor RPM (BDShot)	flash **Bluejay** onto the BLS 60A (ESC Configurator, over the FC's passthrough), then `SERVO_BLH_BDMASK 15`; the ESC tab shows four RPMs at idle	the board's timers already support it (`check_firmware.py bdshot`); BLHeli_S as shipped never answers, which is why the mask ships 0. Once RPMs read true, `INS_HNTCH_MODE 3` for the RPM notch. `J2.8` (`ESC_TEL`) stays receive-only and inert on this ESC
3.8	**Motor order and direction**	motor test, **props off**	`J2.3`–`J2.6` = M1–M4
3.9	Buzzer, LEDs	arming tone, strip lights	
3.10	TFS20-L, if fitted	tape measure at 1 / 5 / 10 m	`RNGFND1_TYPE 46`
3.11	Compass	calibrate, away from the ESC and the pack	it lives in the GPS module — mount it as far from the 60 A ESC as the frame allows

#	Check	How	Note
3.12	Tuner (U13) PLL lock	0x60 on I2C2; tune to 1626 MHz and read the lock-detect bit; a meter on C56 (VTUNE)	VTUNE should sit between the rails, not pinned to 0 V or 3.3 V. Pinned means the loop filter is not doing its job. The driver must set the charge pump to 600 µA (CPS=0, ICP=0): the loop filter keeps 39–76° of phase margin there, but only 23° at 1200 µA
3.13	Antenna feed monitor	multimeter on U25.1 (PC2_C), GCS text, SOA log	open: ≈0 V (<3 mA); OK with HC610: ≈1.65 V (15 mA); short: saturated ∼3.3 V (≥28 mA). Safe retry test with resistor (Section 10.1)
3.15	Remote ID, once fitted	module on J6; CAN_P1_DRIVER 1, CAN_D1_PROTOCOL 1, DID_ENABLE 1, DID_CANDRIVER 1, then the CAA operator ID	a phone's Remote ID app sees the beacon. Shipped off: a configured but absent module blocks arming. UK rule from 1 Jan 2028

10.1 Antenna feed monitor bench test

The active antenna bias feed (U24, U25) supplies +5 V on J12 MMCX to power the HC610's internal LNA. Current is sensed by U25 (INA180A2, ×50) across R62 (2.2 Ω) and digitized by ADC3 on PC2_C. Verify all three operational states on the bench:

1. **Open circuit (no antenna connected):** Probe U25.1 (ANT_CURRENT) with a DMM: output voltage sits near **0 V** (<25 mV, corresponding to <0.25 mA). The GCS displays SoOP: antenna open (0 mA) once on transition, and the DataFlash SOA log records state St=1 (OPEN) with mA under 3.0.
2. **Normal active load (HC610 antenna connected):** Plug the HC610 helical antenna into J12 via the RG178 pigtail. The LNA draws nominal 15 mA. U25.1 reads approximately **1.65 V** (15 mA × 2.2 Ω × 50). The GCS reports SoOP: antenna ok (15 mA), and the SOA log records St=2 (OK) with En=1.
3. **Short-circuit detection and automatic retry:** Do *not* dead-short the coax centre pin directly to shield with a screwdriver. Instead, connect a **100 Ω or 50 Ω 1/4 W test resistor** across an SMA pigtail on J12 (drawing 50–100 mA, well above the 28 mA threshold). U25.1 saturates near **3.25–3.30 V**. After the 0.5 s settle filter, the GCS reports SoOP: antenna shorted (retry in 5s), and SOA records St=3 (SHORT). After 5 s of continuous fault, the firmware de-asserts PC5 (En=0), turning U24 off for 10 s (retry_off_s) to protect the switch from dissipation. Disconnecting the test resistor allows the feed to recover to OK or OPEN on the next retry.

Everything optional ships disabled in defaults.parm and must be turned on here. That is deliberate: a rangefinder configured but absent fails prearm, and a bare board arms out of the box. Do not enable a sensor you have not fitted.

11 Part 10 — T3a: thermal, and the board measures half of it for you

U19 (TMP119) measures U9 for you. It sits 3.9 mm from U9 on I2C1 at 0x48, and defaults.parm ships TEMP_LOG 1, so every flight writes a TEMP message to the card.

Part	Why it matters	Status
U9 (3.3 V LDO)	**linear** regulator dropping 1.7 V in SOT-23-5 with no thermal pad, carrying the MCU, flash and microSD	junction **104–157°C** against a **150°C** limit. It has thermal shutdown — being wrong means the 3.3 V rail switching off in flight, not smoke
U8 (+5 V buck)	**16.8 V → 5 V at 0.74 A** through a TI LMR33630ARNXR (VQFN-12), swapped in place 2026-09-20, sandwiched between a 60 A ESC and a battery	junction **70–87°C** against a **125°C** *recommended* max — 38°C of margin at the worst corner; **still an estimate until a thermocouple says so**
U20 (payload buck)	the same LMR33630A carrying +5V_PAYLOAD: servos, the VTX/camera on J22, the lidar, the LED strip and the DroneCAN node	junction **107°C** at the worst drone mission against **125°C** — under 30°C of margin, so **probe it under that mission's load**

Both brackets exist because the junction-to-ambient figure is a property of *your* assembled board and the two ends come from different sources (JEDEC and EVM columns of the datasheet).

Why U8's bracket read 70–139°C until 2026-09-20, and what changed. The load it was modelled at, 0.95 A, was a hand-copied budget that still counted the deleted PMW3901/VL53L1X, the DNP CAN transceiver and an eight-sensor ToF ring that the LD06 replaced — 281 mA of parts not on the board — while missing the tuner's 100 mA on U10. design.LOADS_5V now derives the rail from the 3.3 V load lists and the fitted sensors: **0.74 A** continuous. The 139°C was retired as a budget error, and the part was *then* changed — two steps worth keeping apart. The LMR33630 in its 8-pin HSOIC-8 package was measured against the artwork on 2026-09-19 and its courtyard is 0.80 mm too wide for U8's pocket in every orientation (docs/KNOWN-ISSUES.md keeps those numbers); the *same silicon* in the 12-pin RNX VQFN has a 3.9 × 2.9 mm courtyard, and it was swapped in place on 2026-09-20, taking the worst corner from 117°C to **87°C**.

The rule that keeps it there: nothing that is *payload* draws from this buck. The recording camera, an FPV AIO, the LD06, the servos and the CAN node run from the payload buck U20 on +5V_PAYLOAD. The one exception is the Iridium antenna's 15 mA, which the antenna feed U24 takes from U8's +5 V. check_order.py modules fails any module that is neither in design.LOADS_5V nor marked BEC-powered, so a load cannot land on U8 unbudgeted.

Half of that bracket is now measured rather than assumed. Neither package has a case to bolt a heatsink to, so every watt leaves into whatever copper is attached — and tools/thermal_vias.py reads that copper off the board for *both* regulators. Measured 2026-09-20, after the swap: U8.1 has **1 via within 1.7 mm holding 7600.3 mm^2 of GND plane**, U9.2 has 2 vias onto the same plane, and **no legal position exists for another via on either part** (the search runs to 1.7 mm and refuses in-pad vias, which would wick solder). The RNX is a leadframe package with *no* separate exposed pad, so that plane is the whole thermal path — which is what the datasheet's $R\theta_{JB}$ of 23.3 °C/W is describing. **It still does not license a number** — copper moves the bound monotonically, and only a thermocouple converts it into degrees.

What to do: power it stack-assembled, logging to the card, ten minutes at steady state. Read TEMP[0]. If you own a probe, put it on U8 too and on U9's case once — U19 is 17.2 mm from U8 and cannot speak for it, and one case reading establishes the offset that U19 then carries forward alone.

Stop and ask if TEMP[0] exceeds about 85°C or U8's case exceeds about 100°C.

12 Part 11 — T3b: the RF chain, and the single open question

This is the one place the project is honest about not knowing.

What exists. The whole Iridium chain is written in C and gated against simulation, and the aircraft runs the same C (AP_SoOP, installed by tools/install_soop_backend.py). The burst DSP

finds 95% of bursts at 55 dB-Hz and above with a p90 carrier error near 2 Hz and no false detections (`tools/check_soop.py dsp`). The navigation holds a median of about 200 m and a p95 of about 710 m horizontally through 15–30 minute GNSS-denied flights at 15 m/s on the real constellation, with orbits one to three days old — and the accuracy it reports holds about 97% of the time, which is what the flight EKF needs (`tools/check_soop.py nav`). What limits it is the flight EKF's velocity once GNSS is gone, not the orbits: at 1 m/s of velocity error the median is 200 m; at 0.1 m/s, 36 m.

What is not known is what the real sky and this antenna deliver: how many bursts per minute, at what C/N0, how far each satellite's transmitter is off frequency, and how the TCXO drifts in flight. The simulation's assumption for each is written in `tools/soop_scenario.py`. T3b replaces them with measurements, in three steps, and the filter's settings (`soop_nav_default_cfg` in `firmware/soop/soop_nav.c`, or `APM/SOOP/soop.cfg` on the card) are where the answers go.

12.1 Step 1 — record the sky with the bench kit (no board needed)

Kit A2: the RTL-SDR, the HC610 antenna outdoors with open sky, and the Pi. The antenna's LNA takes its 15 mA from the RTL-SDR V4's bias tee (`rtl_biast -b 1`), so the bench chain is the antenna and one cable.

```
bash sitl/tle/fetch_iridium_tle.sh                          # today's orbits
date -u +%Y-%m-%dT%H:%M:%S.%NZ > rec.time
rtl_sdr -f 1626333333 -s 2000000 -g 40 -n 1200000000 rec.cu8    # 10 min, 2.4 GB
python3 tools/soop_bench.py rec.cu8 --start $(cat rec.time) --site LAT LON H
```

With the ADALM-Pluto instead: the HC610's LNA takes its 15 mA through an SMA bias tee (`RF+DC` to the antenna, `RF` to the Pluto, 5 V from USB on its DC pin — check 5 V on the `RF+DC` centre pin and none on `RF` before the Pluto goes on). The Pluto's crystal is ±25 ppm, up to ±40 kHz at 1626 MHz, so let it warm up for 20 minutes and let `soop_bench.py` find its error first:

```
rx_sdr -d driver=plutosdr -f 1626333333 -s 2000000 -g 40 -F CS16 rec.cs16
python3 tools/soop_bench.py rec.cs16 --format cs16 --gain auto --lo-search \
    --start $(cat rec.time) --site LAT LON H
```

`-site` is where the antenna was, in degrees and ellipsoidal metres (a phone's GPS is close enough). The tool converts the recording to what the board's ADCs would capture (`firmware/soop/iq_import.c`), runs the flight DSP and the flight navigation over it, and reports:

- **Bursts per minute and their C/N0.** The simulation assumes about one burst a second from each satellite in view, at 45–65 dB-Hz. A few per minute is a gain or antenna problem, not a navigation one.
- **The anchored run**, where the filter is told the site. Most bursts should be fused and the **NIS** should sit near 1: that says the filter's noise model fits the real sky. Well above 1, the first suspect is the satellites' transmit offsets, printed beside it: put their rms in `soop.cfg` as `beta_sigma`.
- **The offset run**, started 5 km away. It shows the filter walking in to the site. Its reported accuracy is optimistic — this is a blind start, which the aircraft never uses (it anchors on GPS first).
- **gr-iridium's count**, if it is installed: the DSP should find most of the bursts gr-iridium finds in the Ring Alert band.

12.2 Step 1b — self-interference, with the bench kit on the aircraft

`tools/check_order.py rf` (`fly.rfbench`) gates ready-to-fly on four measurements in `fab/rf-bench-results.json`. Start the file with `python3 tools/check_order.py rf -template > fab/rf-bench-results.json`. Fit the HC610 in its flight position on the tower, run its pigtail to the RTL-SDR on the ground beside the aircraft, and take the four steps in one session, in this order, without moving the antenna:

1. **baseline**: aircraft unpowered;

2. **board_powered**: battery in, motors off;
3. **motors_spinning**: props off, tethered, about hover throttle;
4. **elrs_tx**: as above, with the ELRS link transmitting.

For each step, record five minutes with Step 1's `rtl_sdr` line (`-n 600000000`), then run `soop_bench.py`. Its *DSP: N bursts, n/min* line gives `bursts_per_min`. For `noise_floor_dbm`, run `rtl_power -f 1616M:1626.5M:10k -g 40 -i 10 -e 60 floor.csv` and take the median bin. It is relative dB, not calibrated dBm, but the gain is the same in all four steps, so the steps can be compared. Fill in `utc` and `sky` (clear, or the cloud cover), then run `python3 tools/check_order.py rf`. A step passes if it keeps at least half the baseline's bursts. Because the steps are cumulative, the first step to fail names the culprit: the board's switchers, then the ESC and motor leads, then the control link.

12.3 Step 2 — the board on the bench, after T3

On the SD card: `APM/SOOP/iridium.tle`, a copy of `sitl/tle/iridium-next.tle` fetched that day, and optionally `APM/SOOP/soop.cfg` (`key value` lines: `beta_sigma`, `clkd_q`, `bbg`, `swap_iq`, `replay`, `dut1`, `lo_offset_hz`, ...). Set `GPS2_TYPE 27`; the task starts with the SoOP GPS backend. The board writes `APM/SOOP/clock.dat` itself: the TCXO's offset per degree of IMU temperature, learnt while GPS is good. With it, the next boot skips the clock search (the GCS says *SoOP: clock N Hz from T C*); a stale or wrong table is noticed and dropped on its own. Delete it after changing `Y2`.

1. **Tuner lock.** At boot the GCS shows *SoOP: tuner locked, VCO n*, or an error naming the VCO and the VTUNE ADC state (row 3.12). The LO is set from the 25 MHz TCXO in 23.8 Hz steps, and the filter is told the one actually programmed.
2. **CPU, before any RF.** Copy `board.iq` from step 1's output folder to `APM/SOOP/replay.iq`, add `replay 1` to `soop.cfg` and reboot. The log's `SOS.CPU` is the DSP's share of real time on the H743; keep it well under 50%. Remove `replay` afterwards.
3. **Antenna feed.** The GCS shows *SoOP: antenna ok, 15.0 mA* with the HC610 on `J12`; *open* or *short* is the coax or the antenna, not the sky (Section 6.5). `SOA` in the log has it once a second.
4. **The noise step — the RF path's first sign of life.** Boot with `ant_feed 0` in `soop.cfg`, note `SOS.Nse` after a minute, then reboot with the feed on: it should rise about 19 dB (16–23), from the MAX2112's own noise to the HC610's (`check_soop.py gain`). Much less is a dead LNA or a broken RF path, whatever the sky is doing.
5. **Live capture**, outdoors, the HC610 on `J12` through its pigtail; the board feeds its LNA. `SOS.Bps` (bursts per second) should be comparable to step 1. If the board hears nothing the bench kit hears, try `swap_iq 1` (swapped I and Q mirror the spectrum); if `SOS.Ovr` counts up, the DSP is not keeping up with the 500 kSPS it is fed. The ADCs run at 1 MSPS, the most `PA4`'s slow channel allows, averaged in pairs.
6. **Spurs at the ADC.** Antenna off, `J12` terminated in 50 Ω, battery in: look at `TP9` and `TP10` on a scope or a spectrum analyser. The two LMR33630As run at 340–460 kHz; a harmonic that reaches tens of millivolts there is in the DSP's band. The DSP learns a steady tone and ignores it within seconds, but if one is that large, move Ring Alert away from it with `lo_offset_hz` (20–150 kHz, either sign). `SOS.AErr` counts ADC overruns; it should stay at 0.
7. **The board against the bench kit, on one antenna.** A 2-way splitter after the HC610, one leg to `J12` (it feeds the LNA; the RTL-SDR's bias tee off), the other to the RTL-SDR. Record step 1's `rtl_sdr` line while the board logs, then run `soop_bench.py` on the recording: per satellite, compare its burst count, C/N0 and carrier sigma with the board's `SOB`, and gr-iridium's Ring Alert count with `SOR`, the headers the board decoded itself. The board should hear at least what gr-iridium does in the Ring Alert band: it searches where each tracked satellite is predicted to be with a lower threshold than a blind search can afford.
8. **Static with GPS, 30 minutes at a known spot.** `SOF.HAcc` settles to tens of metres (anchored on GPS) and `SOF.NIS` near 1. `SOS.Clk` is the TCXO: how fast it drifts over those 30 minutes sets `clkd_q` (the default, 0.08 Hz/s/$\sqrt{\text{s}}$, assumes the board temperature wanders by about 3 °C).

12.4 Step 3 — in flight

With `GPS2_TYPE 27` the Iridium fix is the second GPS and `GPS_AUTO_SWITCH 1` moves to it when the real one loses its fix. When denied, the EKF must take position from GPS (`EK3_SRC2_POSXY 3`) and velocity from flow (`EK3_SRC2_VELXY 5`) — not from the Iridium fix, whose velocity is the EKF's own, corrected. Fly first with GPS healthy and compare `SOF` against `GPS` in the log; then a denial test in a safe area.

Vibration on the TCXO. Log five minutes on the ground with the motors off, then a five-minute hover, and compare `SOB.Inn` (each fused burst's innovation) between them. The spread should not more than double. `check_soop.py dsp` passes 2 g at 1 ppb/g on the stack's grommets, but 3 g at 2 ppb/g near 60 Hz fails it: if the hover spreads the innovations out, soften the mount before trusting the fix.

What the aircraft will not do. It offers no Iridium fix until it has had GPS (or a surveyed start within 50 m): started blind, the filter linearises about a guess kilometres out and in simulation reported up to four times better than it was.

Spoofing. From arming, a second copy of the navigation runs without the GPS, on Iridium alone (`soop_guard.c`), and every GPS fix is checked against it before the navigation learns from it. A fix that jumps is refused at once; a GPS that stays further from the Iridium-only position than 3 times its accuracy (and at least 150 m) for 5 s is declared spoofed. The GCS then shows *SoOP: GPS spoofed, N m from Iridium - using SoOP*, the navigation drops the real GPS for the rest of the flight, and the autopilot is moved onto the Iridium receiver (`GPS_AUTO_SWITCH 0`, `GPS_PRIMARY 1`, not saved: a reboot restores them). `SOS.G` logs the guard's state and `SOS.GD` the distance it measured.

In simulation (`tools/check_soop.py nav`) it raises no alarm on honest GPS — including GPS coming back after a 15 minute denial — and catches a 500 m jump in 1–2 s, a 10 m/s drag in about 3 minutes, a 5 m/s drag in 4–5 and a 2 m/s drag in 19 to 24; by then the spoofer has moved the GPS 1.4–2.9 km. A spoofer that drags the position drags the EKF's velocity too, and the Iridium-only copy is fed that velocity, so it follows part of the way. A slower drag may not be caught within a flight. Watch `SOS.GD` in flight: a distance that keeps growing while the GPS looks healthy is the early sign.

A velocity the spoofer cannot drag. With `ind_vel 1` in `soop.cfg`, the Iridium-only copy takes its velocity from the flow sensor and the downward rangefinder instead, whenever both are healthy (flow quality 100 or better; `SOV` logs it). In simulation, with that velocity good to 0.3 m/s, a 10 m/s drag is caught in 20–30 s, a 5 m/s drag in about a minute and a 2 m/s drag in about two, the GPS by then only 260–370 m off, and still no alarm on honest GPS. It ships off: a PMW3901-class sensor is unreliable over grass, so turn it on where the ground has texture, after checking `SOV` against `GPS` in a log.

12.5 TLEs and time

Before each flying day run `bash sitl/tle/fetch_iridium_tle.sh` and copy the result to `APM/SOOP/iridium.tle`. It fetches CelesTrak's supplemental sets for Iridium, fitted to Iridium's own ephemeris and marked by a `C` in column 8 of line 1; if those are unavailable it says so and writes the public TLEs instead, which are several hundred metres further off along-track. The filter weighs each satellite by which kind its elements are and their age, and ignores any older than 7 days. If IERS Bulletin A gives |UT1−UTC| above 0.1 s, put it in `soop.cfg` as `dut1`: 0.1 s turns the satellites 46 m.

The SITL scenarios (`sitl/`) still fly the synthetic `DopplerErrorModel` of `sitl/soop_link.py`, not this chain, so do not read a position-hold verdict off them either.

13 Part 12 — T3c: IMU temperature calibration

Free, and it attacks the dominant error term: inertial drift is quadratic and dominated by attitude error leaking gravity into the horizontal channel, and gyro and accelerometer bias are strongly temperature-dependent in a stack with almost no airflow. A Pixhawk FMUv6X holds its IMUs warm with heaters; this board has none (the respin's heater could not sit near the IMUs, so it was removed), so ArduPilot's software correction, `INS_TCAL*`, is the whole of the temperature story. It is deliberately *not* in

`defaults.parm`: it is a learning mode, not a flight setting.

1. Run a full six-axis accelerometer calibration first — it sets the `INS_ACC*_CALTEMP` and `INS_GYR*_CALTEMP` the fit is referenced to.
2. Cool the board below its minimum operating temperature: a domestic freezer, in a sealed bag against condensation.
3. Set `INS_TCAL1_ENABLE 2` (**2 = start learning**, not 1) and `INS_TCAL1_TMAX` to the target; the same for `INS_TCAL2_*` — there are two IMUs.
4. Power it up in a warm room and **do not let it move until the run completes.** Movement invalidates the fit silently.
5. Let it rise through at least 25°C of range, freezer to desk, allowing ten minutes per temperature point. Then verify the fit with the offline plotting tool: the corrected traces should sit near zero across the whole range.

Do this with the **stack assembled**, not the board in free air, because the assembly *is* the thermal environment.

14 Part 13 — T3d: the pads and test points

All **10** pads and test points print their function (`L5V`, `DIN`, `BZ`, `B5V` ...) at 1.0 mm, each placed where it is legal silkscreen *and* sits clearly nearer its own pad than any other part — `tools/check_board.py silk_owner` gates that, because a label that reads as its neighbour's is worse than a blank pad: it will be trusted. Preflight fails a pad left blank. `docs/img/padmap.svg` and the assembly drawings name them too.

Three of them are wire pads — the LED strip's supply and ground (`P151`, `P153`) and the buzzer's supply (`P161`). Check each against `docs/img/padmap.svg` before soldering; never identify a gold square by counting from its neighbour.

15 Part 14 — T4/T5: tethered, then flight, strictly in this order

15.1 Foam, shroud and coating, after the bench and before the first flight

- **Baro foam** (`design.FOAM`): a 3 mm open-cell block over `U4` (MS5611) on the board's underside. It keeps light, prop wash and the ESC's warm air off the port without sealing it. Glue only its edge.
- **IMU shroud**: a 1.5 mm adhesive closed-cell pad on the ESC's top face, under `U2`, `U3` and `U4`. `check_mech.py mechanical` proves both leave over 1 mm of air in the ESC gap; neither may touch the board.
- **Conformal coating**, only after T3 passes: acrylic with a UV tracer, inspected under a UV torch. Mask first: every connector mouth (`J1`–`J23`), the microSD socket `J8`, the MMCX `J12`, `SW1`/`SW2`, the Tag-Connect pads `J19`, every test and solder pad (`docs/img/padmap.svg`), the four stack holes, and `U4`'s baro port — coat that and the barometer reads the coating. Coat both sides, let it cure, then refit the foam.
- **A foam cap on `Y2`**, the tuner's TCXO on the underside: 3 mm open-cell, edge-glued, not pressed on the part. It is against sudden steps and prop wash, not slow drift, which the filter absorbs (`check_soop.py sweep`). Fit it after T3b step 2, so the bench measures the bare part first.
- **Kapton over the microSD card** (`J8`): the catalogue, `soop.cfg` and the dead-reckoning applet are on it, and vibration walks a push-push card out.
- **Neutral-cure RTV** on `L2`, `L5`, `J12` and `J1` — never on `Y1`, `Y2` or the IMUs, because a stressed crystal changes frequency.

15.2 Staking the plugs

Before the first tethered hover, check how each plug is held, because a 7-inch quad's vibration walks a friction plug out over a few flights. On this board nothing needs a bead of silicone:

- every JST-GH port — `J2`, `J3`, `J5`, `J6`, `J9`, `J11`, `J14`, `J21`, `J22` — latches: check each one clicks, and pull gently on the lead to prove it;
- the MMCX on `J12` snaps on with a detent; tie the coax down 10–15 mm from it;
- the servo plugs on `J17`/`J23` are a friction fit on SMD pads, held by the printed clip over both — fit the clip before flight. On the lander, where the leads take load, add an epoxy fillet along each header body too, then pull each lead sideways by hand: nothing may move. (Through-hole headers would land their pins on `J5`/`J11` underneath.)
- the LED strip and buzzer are soldered.

`J1` (USB) and `J19` (SWD) are bench ports: unplug them before flight. `tools/check_board.py connectors` fails if a connector is added without saying how it stays in, or if one declared staked is missing from this list.

15.3 Order of flights

1. **Run the SITL scenarios first:** `bash sitl/run_scenarios.sh`. It found three prearm faults no build check saw.
2. **Battery via the smoke stopper and the EC5-to-XT60 adapter** the first time.
3. **Tethered hover in Stabilize.** Land. Pull the log.
4. Tune `INS_HNTCH_FREQ` and `INS_HNTCH_BW` from the log's FFT.
5. **GPS Loiter.** This is a working aircraft.
6. **Props on last**, after motor direction is confirmed.

Everything past GPS Loiter is research. GNSS-denied position hold is uncharacterised until T3b is measured.

15.4 T5b — the velocity the aircraft really has

The simulations assume the EKF's velocity drifts 1 m/s once the GPS is gone. Measure it instead, with no new hardware:

1. Set `LOG_REPLAY 1` and `LOG_DISARMED 0`, do T3c (`INS_TCAL`) first, and fly three 10-minute patterns on GPS — 15 m/s legs, turns, a hover — on a calm day and a windy one.
2. Build ArduPilot's Replay (`./waf replay` in the tree `tools/build_firmware.sh` uses) and re-run each log with the GPS taken away 5 minutes in. The replayed EKF's velocity against the logged GPS velocity is the real drift: its size and how long it holds one way.
3. Put those into `python3 tools/check_soop.py sweep` (`-vel-sigma`, `-vel-tau` on `soop_scenario.py obs`) for the metres this aircraft would hold.
4. Tune drag fusion from the same logs — `EK3_DRAG_BCOEF_X/Y` start at mass over frontal and side area, `EK3_DRAG_MCOEF` between 0.1 and 1.0, until the wind estimate holds steady across the legs — and replay again. Ship the values only once measured.

> **Operational rule while no flow camera is fitted.** EKF source sets 2 and 3 use optical flow for `VELXY`, so today they have **no velocity source**. They are reachable only by the RC switch on `RC9_OPTION 90`: **never select source set 2 or 3 in flight.** Set 1 — GPS, which is also how a SoOP fix will arrive — is the only one with data behind it. And do not "fix" `FLOW_TYPE 5` to 0 with those sets configured: ArduPilot validates every configured set at arm time, and that would ground the aircraft.

16 Part 15 — Audit: what is still manual, soldered, or unmeasured

This is the audit you asked for. It is split into the three kinds of risk, because each is removed by a different action: *manual work* by buying a jig or an alternative, *hand soldering* by a stencil or an assembly tier, and *unmeasured values* only by a bench session.

16.1 15.1 Manual soldering

The board is machine-assembled, so the soldering is deliberately minimal. What remains:

Item	Joints	Risk and mitigation
Buzzer, `P161/P162`	2	low. Must be a **passive** piezo; fitting an active one costs you every arming and failsafe tone pattern, which you will only notice in the air
LED strip, `P151–P153`	3	low, and optional
`U3`, second IMU	14-pin LGA	**only if you chose Economic.** An LGA-14 has no accessible leads, so it needs hot air or a stencil, not an iron. Standard assembly fits it and removes this entirely

The inefficiency, stated plainly: choosing Economic to save one part moves an LGA-14 onto your bench and does *not* save the feeder fees. It is the single manual soldering task on this board that needs real equipment, and it buys you one optional IMU. Unless you want the redundancy, Standard is the cheaper answer once your time is counted.

There is **no stencil requirement** and no hand-placed passives. That is by design.

16.2 15.2 Manual assembly

Task	Why it is manual, and what would remove it
The stack	four bolts, four spacers, grommets, the ESC and the frame's press nuts. Not automatable and not worth automating
Motor screws	16 screws whose length depends on a tapped depth you must measure. The M3 assortment removes the *reorder*, not the measurement
Skids	printed, not bought. This is a feature: the 3.5 mm thickness is yours to set, which is why the screw length is not a fixed number
Board connectors	the GPS, RC, I2C, lidar and ESC leads all plug in; no crimping except the spare pigtails
The first flash	the H743's ROM DFU needs only USB and the `BOOT` button (Part 8). SWD — the `J19` Tag-Connect pads — is the fallback, not the route
The GPS-loss applet	`copter-deadreckon-home.lua` must be copied to the microSD by hand; the build does not package it into the `.apj`
Companion mount	no mount exists yet. Deferred with the companion, so nothing blocks

One structural constraint worth naming, because it is not removable now:

- **The board is 46.5 × 47.2 mm, below JLCPCB's 70 × 70 mm Standard minimum.** Confirmed against their capability page: Standard single-PCB assembly runs 70 × 70 mm to 460 × 500 mm, while Economic runs 10 × 10 mm up. So Standard needs the board panelled with breakaway rails — an extra fabrication step and an extra thing to confirm in their preview, not a

blocker. Economic avoids the panel entirely but charges the feeder fees anyway and drops an LGA onto your bench (Section 15.1), which is why Standard plus panelisation is the recommendation.

16.3 15.3 Measurements that have not been confirmed

This is the longest list, and every row is a value some part of the build *assumes*. The gate's own verdict names six of them as `NOT READY TO FLY`; three more were closed outright on 2026-09-20 by evidence (Section 3.2).

Unconfirmed value	What it decides	Confirmed by
Motor boss tapped depth	M3 × 14 vs 12 + washer. Too deep a screw clamps nothing while feeling tight	your calipers, before the first of 16 screws. Covered by the assortment
Press-nut tapped depth (30.5 mm joint)	M3 × 35 vs 33 + washer, on 8.0 mm of engagement	your calipers
Skid thickness, **assumed** 3.5 mm	sets the motor screw length entirely	you, because you print them. Measure the print
Grommet flange ≤ 6.0 mm	the nearest copper is 3.32 mm from a hole centre; a 6.0 mm flange clears it and a 7 mm washer overlaps by 0.18 mm (nylon, so no short)	calipers on the grommets you receive
ESC lead continuity	whether the GH 8P to SH 8P lead is straight-through. Wrong here means a dead board or a smoked one	a multimeter, both ends, **before any VBAT**
`J3` plug	whether the received GPS lead's boot seats	fit the plug on arrival
`BATT_AMP_PERVLT`	battery failsafe and consumed-mAh. Ships 25.0; it is a property of the ESC's shunt, not of this board	a known load at T3.5
IMU orientations	`ROTATION_YAW_180` (`U2`) / `ROTATION_YAW_90` (`U3`), worked out from the pads and re-checked by `check_firmware.py hwdef`, never measured. A mirrored axis means it does not fly	roll and pitch it, watch the horizon (T3.1)
`U8` junction temperature	70–87°C modelled against a 125°C recommended max, on an honest budget *and a swapped part* since 2026-09-20; the worst corner has 38°C of margin. If the board measures well past its bracket the fix is a **respin**	a thermocouple, T3a
`U9`'s board-to-junction offset	makes `U19`'s logged value meaningful rather than a nearby number	one probe reading on the case, then `U19` carries it
`FLOW_ORIENT_YAW`	the sign of the flow correction, if flow is ever fitted	bench calibration
The RF sigma the antenna delivers	**everything** about GNSS-denied hold. Model says 180 m; the boundary was measured at 20 m	bench RF, T3b. **The single open question**

Unconfirmed value	What it decides	Confirmed by
Whether this antenna can produce a usable Iridium fix at all	the project's premise	T3b
Flow quality over real grass	simulated flow is perfect flow	a field test
Radxa Zero 3W hole pattern	the companion tray, if the companion is ever fitted	**not published** — measure the board, then 3D-print a tray
Frame top-plate standoff spacing	nothing to buy: the kit supplies them	the kit
Grommet compression under load	a property of the rubber	the assembled stack

16.4 15.4 The three worst, ranked

1. **The RF sigma (T3b).** It is the only item that can invalidate the project's premise rather than a build step, and no amount of desk work substitutes for it. Everything downstream of it — position-hold claims, the SITL scenarios' hold/runaway results — is explicitly not readable until it is measured.
2. **`U8`'s junction temperature.** 87°C at the worst corner against a 125°C recommended maximum — 38°C of margin, after the 2026-09-20 swap to the LMR33630ARNXR. It is now a number to confirm rather than one to rescue, and the payload BEC is what stops a later addition eating the margin.
3. **The ESC lead and the tapped depths.** Cheap to check, catastrophic to skip. Both are on this list only because they are properties of parts somebody else manufactured, and both are silenced by owning the M3 assortment and a multimeter.

16.5 15.5 Design-level inefficiencies, for a future revision

These do not affect this order. They are recorded because a respin is the only place they can be fixed, and one of them (`U8`) may force one anyway.

- **USB runs the logic but not the payload rail**, and back-feeds about 4 V onto `VBAT` and the ESC's battery pin through `U8` and `Q4`.
- **The payload rail cannot carry every payload connector at once**: 1.48 A is inside `L5`'s 2.1 A, but puts `U20` at about 135 °C against 125 °C (`check_order.py modules`). Each declared mission fits.
- **The Economic variant saves one part and not the feeder fees**, which makes it a worse deal than it reads as (Section 15.1).
- **`U20`'s input decoupling is a corner short**: one 100 nF (`C66`) 2.4 mm from its input pins and the nearest 10 µF (`C18`) 5.8 mm away, where the datasheet's layout puts both within about 1.5 mm. `L5` fills the underside beneath it and the top side is full, so there is no site on this board. It only feeds the payload rail; T3b step 1b shows what its switching noise does to Iridium reception.
- **In4 rail fragmentation** is an accepted warning on this board.

17 Part 16 — What only the outside world can settle

Section 16.3 is the authoritative register of every value this build *assumes*, and it is deliberately not repeated here — an earlier draft restated nine of its rows and the two lists were free to disagree. What

follows is the short list no local check and no desk work can close, because each needs a fab, a supplier, a vendor or a maintainer to answer it.

Item	Status
JLCPCB's upload preview	outline, panel size, CPL orientation, stackup and drills. No local tool substitutes for looking at their render
1 oz outer copper	every current figure assumes it, and 0.5 oz halves every trace's rating. Confirm what actually ships
LCSC stock	a live external fact, age-gated at 7 days. Re-fetched **2026-09-19**: 56/56 codes resolve, covering 2 boards
The 1620 MHz SAW filter	not stocked at LCSC — a separate board from a separate supplier, and the only part of the RF front end this BOM cannot buy
ArduPilot board ID	**7180** is free and below the ceiling; the request, the registry diff and the required `Readme.md` are ready in `firmware/`. The local build still uses 9001, which is above the ceiling the registry asks you to respect

18 Part 17 — What is not built yet

The board and its firmware are green. The project's *premise* is not, and a clean gate says nothing about it, so it is stated here rather than left to the repository's own notes.

The Iridium chain exists in C and is gated against simulation; nothing about it is proven on hardware or real sky yet. That is T3b (Part 11).

Piece	Status
The least-squares Doppler solve	**built and validated** — `tools/soop_solver.py`, the Python reference; it reports *singular* on degenerate geometry rather than inventing a number
SGP4 and the ephemeris	**built and validated** — `firmware/soop/sgp4.c` and `soop_ephem.c` agree with the Python reference to 0.15 m on the whole catalogue, and the frame rotation with Vallado's published example to 16 m. TLEs are read by column, with no `scanf`, because the flight C library has none with floating point
Burst detection and frequency estimation	**built and gated** — `firmware/soop/soop_dsp.c`, against a simulated sky with the board's impairments: 95% of bursts at 55 dB-Hz and above, p90 carrier error about 2 Hz, no false detections. A bench recording converts into the same DSP (`iq_import.c`)
The navigation	**built and gated** — `firmware/soop/soop_nav.c`, a Kalman filter over position, the flight EKF's velocity drift, the TCXO, the GPS time tag, baro, and each satellite's orbit and transmit errors. Over simulated 15–30 minute denied flights: median 200 m, p95 about 710 m; its reported accuracy holds 97% of the time; no burst credited to the wrong satellite in 23 346 (`tools/check_soop.py nav`)
The flight firmware around it	**built**, **not proven on hardware** — `firmware/ardupilot/AP_SoOP`: the MAX2112 driver, ADC1/ADC2 simultaneous I/Q capture (battery sensing moved to ADC3), and the task that runs the gated C and hands fixes to the EKF. It compiles into the NAVCORE firmware; tuner lock, the capture rate and the CPU share are T3b step 2
The route into the EKF	**built** — an in-process `AP_GPS` backend (`GPS_TYPE 27`), fed by `AP_SoOP` on the aircraft
SITL with the real chain	**not built** — SITL still flies a synthetic stand-in fix (`AP_SoOPFix::update_from_sitl`)

The solver also established the receiver's requirement, which was written down nowhere before: **about 5 Hz of burst frequency precision over 40–80 bursts**. An 8.28 ms burst has roughly 120 Hz of raw FFT resolution, so the carrier must be estimated well inside a bin — that is the number T3b is really trying to meet.

19 The one-page order

1. `python3 tools/preflight.py` — must print **READY TO ORDER**
2. `bash tools/make_order_bundle.sh` — regenerate everything
3. Upload `fab/NAVCORE-SoOP-order-bundle.zip`, or the gerbers plus the default BOM/CPL pair
4. 6 layers, 46.5 × 47.2 mm, 1.6 mm, **1 oz** outer, ENIG, 4 mil, via 0.45/0.20, vias **epoxy filled and capped**, Mark on PCB **Remove Mark**
5. 5 bare boards, 2 assembled. Standard panelised 2 × 2 unless you want to hand-fit `U3`
6. Confirm `Y1` = `C2682774` and `Y2` = `C19674258`; refresh the stock snapshot if it is over 7 days old
7. Order the aircraft parts from `docs/PARTS.csv`, **including the M3 assortment**, the GH 8P to SH 8P ESC lead, the GH pigtails, the HC610 with its MMCX to SMA pigtail, the DC block, and the Blog V4 dongle. Do not buy a bare PMW3901 or VL53L1X chip

8. While waiting: prepare the board ID request, print the padmap, settle the board's orientation and `AHRS_ORIENTATION`
9. On arrival: inspect under magnification, buzz the ESC lead, measure the tapped depths
10. T1 current-limited rails → T2 bootloader over USB DFU (`BOOT` + `RESET`) → T3 sensors, props off
11. T3a thermal (`U8` needs a probe) → T3c IMU temp cal → T3d the wire pads, checked on the padmap
12. T3b before any position-hold claim: measure the real sigma
13. SITL scenarios → tethered hover → GPS Loiter. **Props on last**